Analytics Engineering: Session 13

State, Debugging, Deployment & CI/CD

Agenda

Part 1 — State & Selectors

- Understanding dbt State
- `state:modified` selector
- `dbt retry` & result selectors
- `dbt clone`

Part 2 — Debugging

- Reading error messages
- Debugging with compiled SQL
- Common errors (YAML, SQL, platform)

Part 3 — Deployment & CI/CD

- Environments (Dev vs Prod)

Part 1: Understanding dbt State

"State" refers to the artifacts (`manifest.json`) from a previous run.

- **Comparison**: dbt compares your current code to the "state" artifacts.
- **Use Case**: "Only run models I changed."
- **Artifacts live in**: `target/` directory (`manifest.json`, `run_results.json`)

The `state:modified` Selector

The most powerful selector for CI/CD.

```
dbt build --select state:modified+ --state path/to/prod/artifacts
```

- `state:modified`: The model's logic, config, or schema changed.
- `state:new`: The model was just added.
- `+`: Run everything downstream of the change.
- **Result**: Saves time and money by skipping unchanged models.

`dbt retry`

If a job fails halfway through:

```
dbt retry
```

- dbt remembers what failed in the last run (from `run_results.json`).
- It restarts execution *only* for the failed nodes and their dependencies.
- No need to manually type `dbt run --select model1 model2 ...`.

This is a certification exam topic under Domain 8 (Leveraging dbt state).

Result Selectors

Combine with state for precise re-runs:

```
# Re-run only models that errored (and their downstream)
dbt build --select result:error+ --state target/

# Re-run only models that had test failures
dbt build --select result:fail+ --state target/
```

Selector	Matches
`result:error`	Nodes that errored (compilation/runtime)
`result:fail`	Nodes where tests failed
`result:warn`	Nodes with warnings

`dbt clone`

Creates a lightweight copy of your project in the warehouse.

```
dbt clone --state path/to/prod/artifacts
```

- **Zero-Copy Clone**: (On Snowflake/BigQuery) Instant copy of tables.
- **Use Case**: Create a staging environment that looks exactly like production for testing.
- **Certification**: Tested under Domain 4 (Managing data pipelines).

Part 2: Reading Error Messages

dbt errors usually tell you:

1. **What** failed (Compilation or Execution).
2. **Where** it failed (Model name, line number).
3. **Why** it failed (Database error, Jinja error).

Tip: Read from the bottom up.

The `dbt.log` File

Located in `logs/dbt.log`.

- Contains the full history of your command.
- Shows the exact SQL sent to the database.
- Essential for debugging "silent" failures or performance issues.

```
# View the last 50 lines of the log
tail -50 logs/dbt.log
```

Debugging with Compiled SQL

When a model fails to run:

1. Go to `target/compiled/dbt_ie/models/.../my_model.sql`.
2. Copy the SQL.
3. Run it directly in your database (DuckDB CLI or DBeaver).

This isolates whether the error is **dbt logic** or **SQL syntax**.

Certification Domain 3: "Troubleshooting using compiled code"

Common Errors

Type	Examples	How to Fix
YAML	Indentation errors	Check spacing (2 spaces, not tabs)
Jinja	Missing `{% endif %}` or `}}`	Match all open/close tags
SQL	Trailing commas, wrong column names	Check compiled SQL
Platform	"Table not found"	Check `ref()` or `source()` spelling
Circular	Model A refs B, B refs A	Redesign the DAG

Certification Domain 3: "Distinguishing between dbt and database errors"

Troubleshooting DAG Failure Points

When a model fails in production:

1. **Check**: Which node failed? Look at `run_results.json`.
2. **Isolate**: Run only the failed model: `dbt run -s failed_model`.
3. **Debug**: Check compiled SQL and logs.
4. **Fix**: Apply the fix, test locally.
5. **Retry**: Use `dbt retry` to pick up where you left off.

Don't re-run the entire pipeline — use selectors and retry.

Part 3: Environments — Dev vs Prod

Schema	`dbt_dgarcia`	`analytics`
Data	Sample or full	Full
Trigger	Manual (`dbt run`)	Scheduled
Purpose	Iterate and test	Serve stakeholders

CI/CD Workflows

Continuous Integration (CI):

- **Trigger**: Pull Request opened.
- **Action**: `dbt build --select state:modified+`
- **Goal**: Verify changes don't break the build.

Continuous Deployment (CD):

- **Trigger**: Merge to `main`.
- **Action**: Deploy to Production (full or state-based build).

Slim CI

Only build what changed — the most efficient CI pattern.

```
# In CI pipeline (e.g., GitHub Actions):
dbt build --select state:modified+ --state prod-artifacts/
```

1. Download production `manifest.json` (the "state").
2. Compare current code to production state.
3. Build only modified models and their downstream dependencies.
4. Run tests on the affected models.

Saves time and compute — essential for large projects.

Job Scheduling

Automating the production run.

Tool	Type	Example
dbt Cloud	Built-in scheduler	Schedule via UI
GitHub Actions	CI/CD pipeline	On push/schedule
Airflow/Dagster	Orchestrator	DAG-based scheduling
Cron	Simple timer	`0 6 * * *` (daily at 6am)

Typical Production Job: Build all models, run all tests, generate docs, check source freshness.

The dbt Catalog

> *Conceptual — this is a dbt Cloud feature tested on the certification exam.*

The dbt Catalog provides a searchable, browsable view of your project:

- Model descriptions and column-level documentation
- Test coverage and results
- Lineage graph with upstream/downstream visibility
- Data freshness indicators

Locally, `dbt docs generate` + `dbt docs serve` provides a similar experience.

What have we learned in this session

- Use `state:modified` and result selectors for efficient builds
- Retry failed runs with `dbt retry`
- Clone environments with `dbt clone`
- Debug errors using compiled SQL and `dbt.log`
- Distinguish between dbt, SQL, and platform errors
- Configure CI/CD workflows with Slim CI
- Understand deployment concepts and job scheduling

Next Session: Q&A & Comprehensive Review.